SEMAINE 8

Flots normalisants

Changement de variables · Jacobien · RealNVP · Glow

OBJECTIFS DE LA SÉANCE (CE QUE VOUS SAUREZ FAIRE À LA FIN)

1. Expliquer ce qu'est un **flot normalisant** et pourquoi il permet une **vraisemblance exacte**.
2. Appliquer la **formule du changement de variables** pour calculer $p_X(x)$.
3. Comprendre le rôle du **Jacobien** et de son **déterminant** dans le calcul de densité.
4. Décrire les architectures **RealNVP** et **Glow** et leurs spécificités.
5. Lire et relier à la théorie un **fragment PyTorch** de couche de couplage.
6. Expliquer comment entraîner un RealNVP par maximum de vraisemblance et comment générer de nouveaux échantillons.

1 Fil conducteur : de S7 à S8

MISE EN CONTEXTE — DE LA BORNE VARIATIONNELLE À LA VRAISEMBLANCE EXACTE

En S7, le VAE a introduit un espace latent probabiliste et une fonction objectif fondée sur l'ELBO. Cette semaine, nous cherchons une autre voie : construire une transformation entre latent et données qui soit ***inversible****. Cette contrainte permettra de calculer exactement la densité des observations.*

QUESTION CENTRALE DE LA SEMAINE

Comment transformer une distribution simple en une distribution complexe tout en conservant une vraisemblance exacte ?

CE QUE VOUS DEVEZ SUIVRE PENDANT LA SEMAINE

Le fil logique est :

changement de variables $\rightarrow$ Jacobien $\rightarrow$ transformations inversibles $\rightarrow$ RealNVP $\rightarrow$ Glow.

Chaque étape répond à une difficulté de calcul introduite par la précédente.

2 Pourquoi les flots normalisants ?

2.1 Rappel : les modèles génératifs vus jusqu'ici

— **Modèles auto-régressifs (S5) :**

$$p(x) = \prod_i p(x_i \mid x_{<i})$$

vraisemblance exacte, mais génération séquentielle potentiellement lente.

— **Autoencodeurs variationnels (VAE, S7) :**

$$p_\theta(x) = \int p_\theta(x \mid z) p(z)\, dz$$

modèle latent probabiliste, mais marginale généralement difficile à calculer exactement ; entraînement par une borne, l'ELBO.

— **Réseaux antagonistes génératifs (GAN, à venir) :** génération implicite, mais pas de densité $p_\theta(x)$ directement calculable.

2.2 La réponse : les flots normalisants

Les **flots normalisants** sont une famille de modèles génératifs qui :

— partent d'une distribution simple $p_Z(z)$, souvent une gaussienne standard ;
— appliquent une séquence de transformations **bijectives** et **différentiables** ;
— obtiennent une distribution complexe $p_X(x)$;
— permettent de calculer **exactement** la vraisemblance grâce à la formule du changement de variables.

DÉFINITION — FLOT NORMALISANT

Un flot normalisant est une transformation

$$f_\theta : \mathbb{R}^d \to \mathbb{R}^d,$$

bijective et différentiable, qui transforme une distribution simple $p_Z(z)$ en une distribution complexe $p_X(x)$.

DEUX SENS À NE PAS CONFONDRE

Un flot est utilisé dans deux directions complémentaires :

— **Génération :** on tire $z \sim p_Z(z)$ puis on calcule

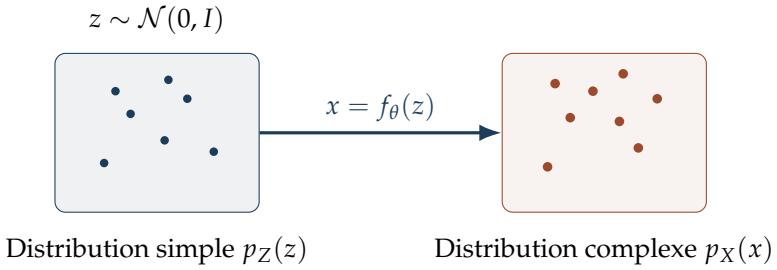
$$x = f_\theta(z).$$

— **Évaluation de densité :** pour une donnée observée x, on calcule

$$z = f_\theta^{-1}(x),$$

puis on évalue sa densité grâce à la formule du changement de variables.

La bijectivité impose ici la même dimension pour z et x. Des extensions plus générales existent, mais elles dépassent le cadre de cette semaine.



Un flot normalisant transforme une distribution simple en une distribution complexe via une fonction bijective.

3 La formule du changement de variables

Avant d'aborder la formule, gardez une idée simple en tête : une transformation déplace les points dans l'espace, mais la masse totale de probabilité doit rester égale à 1. Le facteur jacobien corrige la densité lorsque l'espace est localement étiré ou contracté.

3.1 Cas 1D : intuition

Considérons une variable aléatoire z suivant une distribution $p_Z(z)$ et une transformation bijective, dérivable :

$$x = f(z).$$

DÉFINITION — CHANGEMENT DE VARIABLES EN 1D

La densité de x s'écrit :

$$p_X(x) = p_Z(z) \left| \frac{dz}{dx} \right|, \qquad z = f^{-1}(x).$$

- **INTUITION GÉOMÉTRIQUE**

Supposons que p_Z soit uniforme sur $[0,1]$ et que :

$$f(z) = 2z.$$

Alors :

— l'intervalle $[0,1]$ devient $[0,2]$, donc il est deux fois plus long ;
— la probabilité totale doit rester égale à 1 ;
— la densité doit donc être divisée par 2.

Le facteur de correction est :

$$\left| \frac{dz}{dx} \right| = \frac{1}{2}.$$

EXEMPLE CONCRET : SIGMOÏDE

Soit :

$$z \sim \mathcal{N}(0,1), \qquad x = \sigma(z) = \frac{1}{1+e^{-z}}.$$

Alors :

$$p_X(x) = \mathcal{N}(z;0,1) \left| \frac{dz}{dx} \right|,$$

avec :

$$z = \log\left(\frac{x}{1-x}\right), \qquad \frac{dz}{dx} = \frac{1}{x(1-x)}.$$

3.2 Cas multidimensionnel

Soit une transformation :

$$x = f_\theta(z), \qquad f_\theta : \mathbb{R}^d \to \mathbb{R}^d,$$

bijective et différentiable.

FORMULE DU CHANGEMENT DE VARIABLES

Pour :

$$x = f_\theta(z), \qquad z = f_\theta^{-1}(x),$$

deux écritures équivalentes sont utiles :

$$p_X(x) = p_Z(z) \left| \det J_{f_\theta}(z) \right|^{-1},$$

et :

$$p_X(x) = p_Z(f_\theta^{-1}(x)) \left| \det J_{f_\theta^{-1}}(x) \right|.$$

Ici :

$$J_{f_\theta}(z) = \frac{\partial f_\theta(z)}{\partial z}$$

est le Jacobien dans le sens génératif, tandis que :

$$J_{f_\theta^{-1}}(x) = \frac{\partial f_\theta^{-1}(x)}{\partial x}$$

est le Jacobien dans le sens inverse.

DÉFINITION — MATRICE JACOBIENNE

La matrice jacobienne de f_θ^{-1} est :

$$J_{f_\theta^{-1}}(x) = \begin{bmatrix} \frac{\partial z_1}{\partial x_1} & \cdots & \frac{\partial z_1}{\partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial z_d}{\partial x_1} & \cdots & \frac{\partial z_d}{\partial x_d} \end{bmatrix}.$$

3.3 Interprétation géométrique du déterminant

Raisonnons avec le Jacobien du sens génératif :

$$J_{f_\theta}(z).$$

— Si :

$$|\det J_{f_\theta}(z)| > 1,$$

la transformation étire localement le volume : la densité diminue.

— Si :

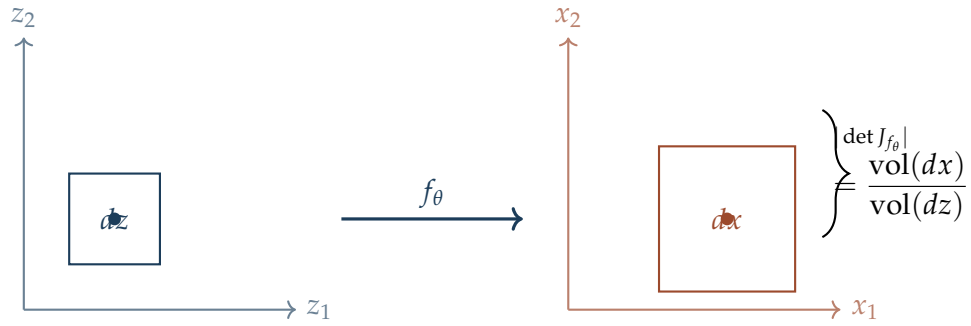
$$|\det J_{f_\theta}(z)| < 1,$$

la transformation contracte localement le volume : la densité augmente.

— Si :

$$|\det J_{f_\theta}(z)| = 1,$$

le volume local est préservé.



Le déterminant jacobien mesure le changement de volume local induit par la transformation.

3.4 Version logarithmique

Pour éviter les problèmes de sous-dépassement numérique, on travaille avec la log-vraisemblance :

$$\log p_X(x) = \log p_Z(f_\theta^{-1}(x)) + \log \left|\det J_{f_\theta^{-1}}(x)\right|.$$

Cette écriture est utilisée pour l'entraînement : pour chaque donnée observée x, on revient vers le latent puis on ajoute la correction de volume.

CONVENTION DE SENS UTILISÉE DANS CETTE SÉANCE

Nous utilisons deux directions :

— **Sens génératif :**

$$z \to x, \qquad x = f_\theta(z).$$

— **Sens densité / entraînement :**

$$x \to z, \qquad z = f_\theta^{-1}(x).$$

La formule de densité utilise le Jacobien de la direction effectivement parcourue. Ainsi, si une couche directe contribue :

$$\log|\det J_f| = \sum_i s_i,$$

alors sa transformation inverse contribue :

$$\log|\det J_{f^{-1}}| = -\sum_i s_i.$$

4 Composition de transformations : le flot

4.1 Principe

Un flot est composé de plusieurs transformations :

$$z_0 \sim p_Z(z_0), \qquad z_1 = f_1(z_0), \qquad z_2 = f_2(z_1), \qquad \ldots, \qquad x = z_K = f_K(z_{K-1}).$$

VRAISEMBLANCE POUR UNE COMPOSITION

Si :

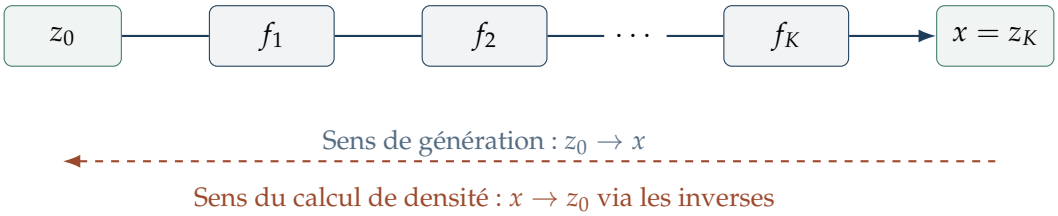
$$z_k = f_k(z_{k-1}), \qquad k = 1, \ldots, K, \qquad x = z_K,$$

alors :

$$\log p_X(x) = \log p_Z(z_0) - \sum_{k=1}^{K} \log\left|\det J_{f_k}(z_{k-1})\right|.$$

Chaque couche ajoute une correction de volume ; les logarithmes transforment le produit des déterminants en somme.

Écriture équivalente : si l'on parcourt les transformations inverses depuis x, les mêmes corrections apparaissent avec le signe opposé.



Composition de transformations : le flot complet. La vraisemblance s'obtient en parcourant les transformations inverses.

4.2 Avantage de la composition

Composer plusieurs transformations simples permet de créer une distribution finale très complexe, même si chaque transformation individuelle reste relativement simple.

ANALOGIE

C'est comme façonner un bloc d'argile : pousser, tirer et étirer sont des opérations simples, mais leur composition peut créer une sculpture complexe.

5 Le défi pratique : calcul efficace du Jacobien

5.1 Le problème

Pour une transformation arbitraire :

$$f : \mathbb{R}^d \to \mathbb{R}^d,$$

la matrice jacobienne possède d^2 éléments, et le calcul de son déterminant coûte généralement :

$$O(d^3),$$

par exemple avec une décomposition LU.

Pour :

$$d = 784,$$

on a :

$$d^3 \approx 480 \text{ millions},$$

d'opérations pour une seule transformation dense.

5.2 La solution : jacobien triangulaire

L'idée clé des flots modernes est de concevoir des transformations dont le Jacobien est **triangulaire**. Dans ce cas :

$$\det(J) = \prod_{i=1}^{d} J_{ii},$$

donc :

$$\log|\det(J)| = \sum_{i=1}^{d} \log|J_{ii}|.$$

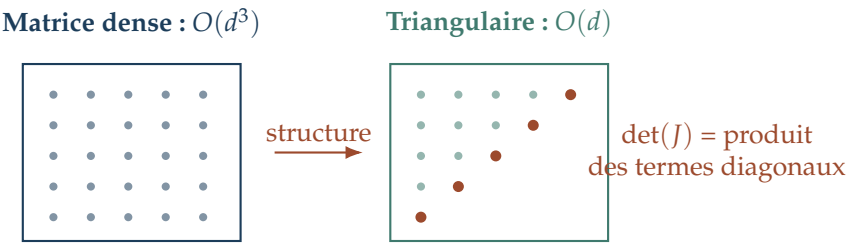
Le calcul du **log-déterminant**, une fois les paramètres de la couche obtenus, devient ainsi :

$$O(d)$$

au lieu de :

$$O(d^3).$$

> NUANCE SUR LE COÛT TOTAL
>
> Le gain en $O(d)$ concerne le calcul du **log-déterminant**. Le coût total de la couche inclut aussi l'évaluation des réseaux qui produisent les paramètres de transformation, par exemple $s(x_a)$ et $t(x_a)$ dans RealNVP.



Les Jacobiennes triangulaires rendent le calcul du log-déterminant tractable.

6 RealNVP : couches de couplage

La difficulté n'est pas seulement de choisir une transformation inversible : il faut également que son déterminant jacobien soit calculable efficacement. RealNVP répond à ce problème avec les **couches de couplage**.

6.1 Présentation

RealNVP (*Real-valued Non-Volume Preserving*) repose sur des couches de couplage affine qui produisent un Jacobien triangulaire.

6.2 Principe d'une couche de couplage

On divise les dimensions en deux parties :

$$x = (x_a, x_b).$$

La transformation directe est :

$$\begin{cases} y_a = x_a, \\ y_b = x_b \odot \exp\big(s(x_a)\big) + t(x_a). \end{cases}$$

Ici :

— $s(\cdot)$ et $t(\cdot)$ sont des réseaux de neurones, par exemple des MLP ou CNN ;
— $\odot$ désigne le produit élément par élément ;
— $\exp(s(x_a))$ garantit une échelle strictement positive.

> DÉFINITION — COUCHE DE COUPLAGE REALNVP
>
> Une couche de couplage transforme une partie des dimensions en fonction de l'autre, tout en laissant cette dernière inchangée.

6.3 Inverse de la transformation

L'inverse est immédiat :

$$\begin{cases} x_a = y_a, \\ x_b = (y_b - t(y_a)) \odot \exp\big(-s(y_a)\big). \end{cases}$$

Cette simplicité permet de passer facilement de x à z et de z à x.

6.4 Calcul du Jacobien

Le Jacobien direct est triangulaire :

$$J = \begin{bmatrix} I & 0 \\ \frac{\partial y_b}{\partial x_a} & \operatorname{diag}(\exp(s(x_a))) \end{bmatrix}.$$

Son déterminant vaut :

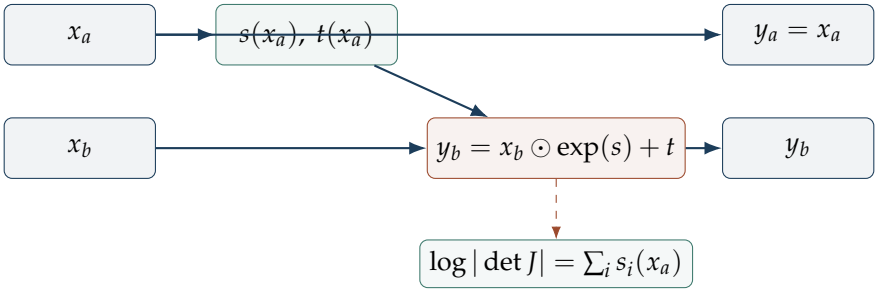
$$\det(J) = \prod_{i=1}^{d_b} \exp(s_i(x_a)) = \exp\left(\sum_{i=1}^{d_b} s_i(x_a)\right).$$

Donc :

$$\log|\det(J)| = \sum_{i=1}^{d_b} s_i(x_a).$$

EFFICACITÉ DE LA COUCHE DE COUPLAGE

Le log-déterminant se calcule en $O(d)$ et ne nécessite pas de construire explicitement la matrice jacobienne complète.



Dans RealNVP, x_a reste inchangé et paramètre la transformation affine appliquée à x_b.

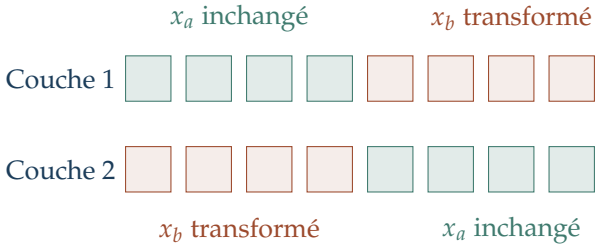
6.5 Alternance des masques

Pour que toutes les dimensions soient transformées, on alterne les partitions :

— couche 1 : les premières dimensions forment x_a et les autres x_b ;

— couche 2 : la partition est inversée ;

— les couches suivantes alternent de la même manière.

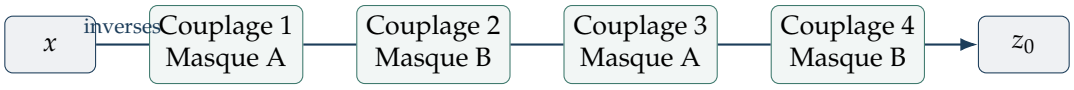


L'alternance des masques permet à toutes les dimensions d'être transformées au fil des couches.

7 Architecture complète de RealNVP

Une architecture RealNVP complète compose plusieurs couches de couplage avec alternance des masques :

$$x = f_K \circ f_{K-1} \circ \cdots \circ f_1(z_0).$$



$$\log p_X(x) = \log p_Z(z_0) + \sum_k \log|\det J_{f_k^{-1}}|$$

Pour calculer la densité d'une observation, on traverse les couches dans le sens inverse.

7.1 Entraînement par maximum de vraisemblance

L'entraînement d'un RealNVP se fait par maximum de vraisemblance exact :

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_X(x^{(i)};\theta).$$

Pour chaque observation x :

1. calculer :
$$z_0 = f_\theta^{-1}(x)$$
en appliquant les inverses de chaque couche ;
2. calculer :
$$\log p_Z(z_0),$$
souvent la log-densité d'une gaussienne standard ;
3. additionner les log-déterminants de chaque couche inverse ;
4. obtenir :
$$\log p_X(x).$$

> AVANTAGE CLÉ
>
> Contrairement au VAE standard, on n'optimise pas une borne variationnelle : pour le modèle de flot choisi, la log-vraisemblance est calculable exactement. Cela permet une évaluation directe de :
>
> $$\log p_X(x).$$

> NUANCE IMPORTANTE SUR LA COMPARAISON
>
> Une vraisemblance exacte n'est pas automatiquement synonyme de meilleure qualité perceptuelle. De plus, les valeurs de log-vraisemblance ne sont comparables proprement qu'avec des données, un prétraitement et une convention de déquantification compatibles.

8 Glow : améliorations de RealNVP

RealNVP rend le calcul exact possible, mais ses masques fixes peuvent limiter le mélange entre dimensions. Glow conserve le principe des flots tout en rendant ce mélange plus flexible.

8.1 Présentation

Glow améliore RealNVP avec des mécanismes qui favorisent l'apprentissage d'un mélange plus flexible des canaux, notamment pour les images.

8.2 Les trois innovations principales

- **ACTNORM : NORMALISATION D'ACTIVATION**

ActNorm est une transformation affine par canal :

$$y_c = s_c x_c + b_c, \qquad x_c = \frac{y_c - b_c}{s_c}, \qquad s_c \neq 0.$$

Pour une carte de caractéristiques de taille $H \times W$, son log-déterminant est :

$$\log|\det J| = HW\sum_c \log|s_c|.$$

Les paramètres sont initialisés à partir du premier mini-lot afin que les activations initiales soient approximativement centrées et de variance contrôlée. Ils deviennent ensuite des paramètres appris indépendants des statistiques des mini-lots suivants.

— **Avantage :** normalisation stable sans dépendre des statistiques du mini-lot à chaque passe ;
— **Inversibilité :** immédiate tant que chaque s_c reste non nul.

- **CONVOLUTION 1×1 INVERSIBLE**

Glow introduit une convolution 1×1 inversible dont la matrice W mélange les canaux à chaque position spatiale :

$$y_{h,w} = W x_{h,w}, \qquad x_{h,w} = W^{-1} y_{h,w}.$$

Pour une carte de taille $H \times W$, la contribution totale au log-déterminant est :

$$HW \log |\det W|.$$

— **Avantage :** mélange appris et plus flexible des canaux ;
— **Inversibilité :** W doit rester inversible ; une paramétrisation LU permet de calculer efficacement le déterminant et l'inverse.

- **COUPLAGES AFFINES**

Glow utilise aussi des couches de couplage affines, similaires à RealNVP, mais souvent paramétrées par des réseaux plus expressifs.

8.3 Architecture d'un bloc Glow



$$\log |\det|_{\text{total}} = \log |\det|_{\text{ActNorm}} + \log |\det W| + \textstyle\sum_i s_i$$

Un bloc Glow combine ActNorm, convolution 1×1 inversible et couplage affine.

8.4 Comparaison RealNVP vs Glow

Caractéristique	RealNVP	Glow
Mélange des dimensions	Masques binaires alternés	Convolution 1×1 apprise
Normalisation	Aucune ou optionnelle	ActNorm inversible
Flexibilité	Limitée par les masques fixes	Plus flexible grâce au mélange appris
Complexité	Plus simple	Plus complexe, généralement plus expressif
Applications typiques	Estimation de densité et images de complexité modérée	Images à plus grande échelle, synthèse et manipulation

9 Du concept au code : couche de couplage RealNVP

Dans une couche de couplage affine, une partie x_a reste inchangée ; elle sert à calculer s et t, qui transforment l'autre partie x_b.

FRAGMENT PYTORCH — TRANSFORMATION DIRECTE

```
s, t = net(x_a).chunk(2, dim=-1)
y_a = x_a
y_b = x_b * torch.exp(s) + t
log_det = s.sum(dim=-1)
```

À observer : on obtient simultanément la transformation et le log-déterminant, sans construire explicitement toute la matrice jacobienne.

FRAGMENT PYTORCH — TRANSFORMATION INVERSE

```
x_a = y_a
s, t = net(y_a).chunk(2, dim=-1)
x_b = (y_b - t) * torch.exp(-s)
```

À observer : le même réseau est évalué sur la partie inchangée. L'inverse ne nécessite aucune itération numérique.

FRAGMENT PYTORCH — GÉNÉRATION

```
z = torch.randn(n_samples, dim)
x = model(z)
```

À observer : après entraînement, la génération suit simplement le sens :

$$z \to x.$$

Pour calculer une vraisemblance, on utilise au contraire le sens inverse :

$$x \to z.$$

10 Avantages et limites des flots normalisants

10.1 Avantages

— **Vraisemblance exacte :** on peut calculer et optimiser directement :

$$\log p_\theta(x).$$

— **Génération parallélisable :** contrairement aux modèles auto-régressifs, les composantes d'un échantillon peuvent généralement être transformées en parallèle à travers les couches du flow. La génération doit néanmoins traverser toutes les transformations inversibles.

— **Inversibilité :** on peut passer de x à z et inversement, utile pour l'interpolation et la manipulation.

— **Espace latent explicite :** on dispose d'une correspondance bijective entre données et latents continus.

— **Densité directement évaluée :** on peut calculer :

$$\log p_X(x)$$

pour le modèle de flot choisi.

10.2 Limites

— **Contrainte d'inversibilité :** toutes les transformations doivent être bijectives, ce qui limite le choix des architectures.

— **Coût :** même avec des Jacobiennes structurées, l'évaluation de nombreuses couches et réseaux auxiliaires peut être coûteuse en grande dimension.

— **Architecture complexe :** concevoir des transformations inversibles à Jacobien tractable n'est pas trivial.

— **Qualité perceptuelle :** pour des images complexes, les flots peuvent produire des résultats moins nets que certains GAN ou modèles de diffusion.

— **Données discrètes :** les flots continus définissent des densités sur des variables continues. Pour modéliser des pixels discrets, on utilise souvent une **déquantification**, qui transforme une valeur discrète en variable continue par ajout de bruit. Cette étape doit être prise en compte pour interpréter correctement les log-vraisemblances.

MESSAGE FINAL S8

Les flots normalisants combinent une densité explicite, une vraisemblance calculable exactement et une génération directe. Cette propriété impose toutefois des transformations inversibles soigneusement conçues. RealNVP et Glow rendent cette contrainte praticable grâce à des Jacobiennes structurées et à des transformations faciles à inverser.

11 Synthèse de la semaine 8 et transition vers S9

SYNTHÈSE — CE QU'IL FAUT RETENIR

— un flot normalisant transforme une distribution simple en une distribution complexe par des transformations **bijectives et différentiables** ;

— la formule du **changement de variables** corrige la densité grâce au déterminant du **Jacobien** ;

— RealNVP rend ce calcul efficace avec des **couches de couplage** à Jacobien triangulaire ;

— Glow améliore le mélange des dimensions avec ActNorm et une convolution 1×1 inversible ;

— contrairement au VAE standard, un flot permet d'évaluer et d'optimiser une **vraisemblance exacte**.

AUTO-VÉRIFICATION

À la fin de cette semaine, vous devriez pouvoir :

1. expliquer pourquoi un flot doit être inversible ;
2. écrire la formule du changement de variables dans le sens $x \to z$;
3. expliquer ce que mesure le déterminant du Jacobien ;
4. dire pourquoi une Jacobienne triangulaire est utile ;
5. expliquer comment RealNVP calcule son log-déterminant ;
6. distinguer RealNVP et Glow ;
7. expliquer pourquoi la déquantification est utilisée pour les images discrètes.

TRANSITION VERS LA S9

"Cette semaine, nous avons conservé une densité explicite et calculable grâce à l'inversibilité. En S9, nous changerons de point de vue : au lieu de normaliser directement une densité, nous apprendrons une ***fonction d'énergie****. Cela introduira les Energy-Based Models, leur fonction de partition et la Contrastive Divergence."*